

WORDPRESS HEADLESS

REST API

REACT + TYPESCRIPT

IMPLEMENTATION MANUAL

bp-tracker

A headless WordPress + React blood pressure tracking application — architecture decisions, requirements, security model, and phase-by-phase implementation specifications.

Version 1.0

Project documentation

Contents

bp-tracker — Implementation Manual

1. Overview
2. Requirements
 - 2.1 Functional Requirements
 - 2.2 Non-Functional Requirements
 - 2.3 User Stories
3. Architecture Decisions
 - ADR-001 — Headless WordPress over a themed WordPress site
 - ADR-002 — REST API over GraphQL
 - ADR-003 — Custom Post Type + Post Meta over a custom database table
 - ADR-004 — Access + refresh JWT over a single long-lived JWT
 - ADR-005 — Vite + React + TypeScript over Next.js
 - ADR-006 — Monorepo over separate repositories
 - ADR-007 — Tailwind CSS, mobile-first
 - ADR-008 — Custom minimal roles over a membership/approval plugin
 - ADR-009 — .env via phpdotenv over wp-env config injection
 - ADR-010 — Categorical X-axis over continuous time-scale X-axis for the trend chart
 - ADR-011 — Vitest over Jest
 - ADR-012 — CI without CD
4. System Architecture
 - 4.1 Component overview
 - 4.2 Data model
 - 4.3 API surface
5. Authorization & Roles Model
 - 5.1 Roles
 - 5.2 Registration & approval flow
 - 5.3 Defense in depth
 - 5.4 Access restriction outside the API
6. Security Model
 - 6.1 Baseline protections
 - 6.2 Hardening measures
 - 6.3 Explicit non-goals / known limitations
7. Testing Strategy
8. Manual Testing Guide
 - 8.1 Backend-only phase (before the frontend existed)
 - 8.2 Frontend-integrated phase
 - 8.3 Roles & approval phase
 - 8.4 Hardening phase
- Appendix A — Prompts Used with Claude Code
 - Phase A — Repository Bootstrap
 - Phase B0 — Local Environment & Plugin Scaffold
 - Phase B1 — Reading Data Model
 - Phase B2 — Authentication
 - Phase B3 — Readings REST Controller
 - Phase B4 — API Documentation

Phase B5 — Backend Continuous Integration

Phase C0 — Frontend Scaffold

Phase C1 — Authentication (Frontend)

Iteration — Session Hardening

Phase C2 — New Reading Form

Phase C3 — History Dashboard (initial version)

Phase C4 — Frontend Continuous Integration

Iteration — Secrets Configuration Revisited

Iteration — Trend Chart Redesign

Iteration — Dashboard Expansion (period selector, zoom, clinical context, summary trends)

Iteration — Visual Redesign

Phase D1 — User Roles & Registration

Phase D2 — Access Restriction Outside the API

Phase — Historical Data Migration

Phase E1 — Rate Limiting

Phase E2 — Token Algorithm Pinning

Phase E3 — SQL Review

Phase E4 — Dependency Vulnerability Scanning

Phase E5 — Safe Error Reporting

Appendix B — Glossary

bp-tracker — Implementation Manual

A headless WordPress + React blood pressure tracking application Architecture decisions, requirements, and phase specifications

1. Overview

bp-tracker is a personal health-metrics logger built to record and visualize blood pressure readings taken twice daily. The project was deliberately scoped as a full-stack showcase: a decoupled WordPress backend (REST API only, no theme rendering) paired with a React single-page application, built with production-grade practices — automated testing, static analysis, CI, and a layered security model — despite its small functional surface.

This document records *why* the system looks the way it does: the requirements that drove it, the architecture decisions made along the way (and the alternatives considered), the system design, the security model, and the exact prompts used to direct implementation of each phase. It is written to be read end-to-end by someone auditing the codebase, or by the author revisiting the project later.

Development process. This project was built in collaboration with Claude Code, Anthropic's agentic coding tool, following a deliberate phase-by-phase workflow: each phase was specified in detail before implementation, then verified (automated tests passing, manual checks where applicable) before moving to the next. Appendix A reproduces every prompt used, in the order they were used, as a transparent record of how the codebase came to be.

2. Requirements

2.1 Functional Requirements

ID	Requirement
FR-1	A user can record a reading with systolic, diastolic (required), and optionally pulse, weight, and free-text notes.
FR-2	A user records readings at least twice daily; the system must handle multiple same-day readings without ambiguity.
FR-3	A user can view their full reading history, most recent first.
FR-4	A user can visualize systolic and diastolic trends over time on a single chart.
FR-5	A user can filter the history/chart by preset periods (7, 10, 30, 90 days), a custom date range, or lifetime (all data).
FR-6	A user can delete a reading.
FR-7	A user can register an account; new accounts require administrator approval before first login.
FR-8	An administrator can approve or manage user accounts without a custom-built interface.
FR-9	A user can only see and modify their own readings — never another user's.

2.2 Non-Functional Requirements

ID	Requirement
NFR-1	Mobile-first UI — the primary usage pattern is a phone, twice a day.
NFR-2	Authenticated API access via token-based auth with real revocation (not a stateless-only JWT).
NFR-3	Automated tests on both sides of the stack (PHP and TypeScript), required for every feature.
NFR-4	Static analysis / linting gates on both sides (PHPStan, WPCS, ESLint, TypeScript strict mode).
NFR-5	Continuous Integration running tests, lint, and analysis on every push.
NFR-6	All code, comments, commit messages, and documentation in English.
NFR-7	The use of AI-assisted development tooling (Claude Code) is transparently documented as part of the project's methodology, rather than concealed.
NFR-8	Baseline security hardening: rate limiting, restricted CORS, dependency vulnerability scanning, least-privilege roles.

2.3 User Stories

- As a **user**, I want to log a reading in under a minute, so that twice-daily tracking doesn't feel like a chore.
- As a **user**, I want to see every reading on one chart — even several from the same day — so I can actually spot trends instead of a cluttered mess.
- As a **user**, I want to filter by recent periods or see my full history, so I can zoom into short-term changes or long-term patterns.
- As a **user**, I want my data private to me, so that no other registered user — even a future one — can see my readings.
- As a **new user**, I want to register an account and understand that it requires approval, so I'm not confused when I can't log in immediately.
- As the **administrator**, I want to approve or manage accounts using tools I already know (the native WordPress users screen), so I don't have to maintain a custom admin UI for something that happens rarely.
- As the **administrator**, I want my account reserved for user management only — never a content author — so the system's trust boundaries are clear.
- As a **user**, I want the app to be comfortable on my phone, since that's where I'll use it in practice.

3. Architecture Decisions

Each decision below follows: **Context** → **Decision** → **Rationale** → **Trade-offs**.

ADR-001 — Headless WordPress over a themed WordPress site

Context: The project's explicit goal was to pair WordPress with a React frontend. **Decision:** WordPress serves exclusively as a backend — a custom REST API — with `show_ui` disabled on the custom post type and no theme rendering involved. **Rationale:** Full separation of concerns; the frontend is free to use its own build tooling, routing, and state management without fighting WordPress's templating layer. **Trade-offs:** Loses WordPress's built-in admin screens for the domain data; requires building custom REST endpoints and a custom auth layer that a themed site would get closer to "for free."

ADR-002 — REST API over GraphQL

Context: WPGraphQL was considered as an alternative API layer. **Decision:** Plain custom REST endpoints under a single namespace. **Rationale:** The domain has one primary resource (readings) with simple CRUD plus one aggregate endpoint (stats). GraphQL's value proposition — avoiding over/under-fetching across many related resource types — doesn't apply to a single-resource API. Adopting it would add resolver complexity, N+1 query management, and a new plugin dependency for no functional gain here. **Trade-offs:** If the domain model grows significantly more relational, this decision should be revisited.

ADR-003 — Custom Post Type + Post Meta over a custom database table

Decision: Readings are stored as a `bp_reading` Custom Post Type with typed post meta fields (`reading_datetime` , `systolic` , `diastolic` , `pulse` , `weight` , `notes`), exposed via `show_in_rest` and `register_post_meta`. **Rationale:** This gets REST exposure, sanitization hooks, and WordPress-native storage essentially for free, with no custom schema migrations to maintain. **Trade-offs:** A purpose-built table would be marginally more efficient at scale; irrelevant at this project's data volume (dozens to low thousands of rows per user).

ADR-004 — Access + refresh JWT over a single long-lived JWT

Context: A JWT, once issued, cannot be invalidated before its expiry — there is no way to implement a real "log out" with a single long-lived token. **Decision:** Short-lived access tokens (15 minutes, HS256-signed, held only in the frontend's memory) paired with revocable refresh tokens (30 days). The refresh token travels only in an `HttpOnly; SameSite=Strict` cookie scoped to the `/auth/*` routes, and is stored **hashed** (SHA-256) in a custom database table (`bp_tracker_refresh_tokens`). Every refresh rotates the token; tokens of one login form a *family*, and presenting an already-used token revokes that whole family. **Rationale:** Enables real logout and revocation, keeps the long-lived credential out of reach of JavaScript (and so of XSS), and makes a stolen refresh token detectable on reuse. Access tokens are checked against a per-user *session generation* counter, so "sign out of all devices", a password change, or an account being made pending again invalidates them immediately instead of at expiry. **Trade-offs:** More implementation surface than a single-token scheme: an extra table, rotation and reuse-detection logic, a CSRF header on the cookie-authenticated routes, and one user-meta read per authenticated request for the generation check. The `SameSite=Strict` cookie also requires the frontend and the API to be served from the same site.

ADR-005 — Vite + React + TypeScript over Next.js

Context: Next.js was considered, given its popularity for React applications. **Decision:** A Vite-built single-page application, not Next.js. **Rationale:** Next.js's core strengths — SSR, SSG, and SEO — don't apply to an application that sits entirely behind authentication with no public-facing pages. Its main potential benefit here (proxying API calls through server-side route handlers to keep tokens in `httpOnly` cookies instead of client-side storage) turned out to be achievable without it: the WordPress API itself sets the refresh token as an `HttpOnly` cookie (see ADR-004). **Trade-offs:** The short-lived access token still lives in the client, in memory only (never in `localStorage`), and is lost on page reload; the app restores the session from the refresh cookie on load. If the auth model moves to fully server-mediated sessions later, this decision should be revisited.

ADR-006 — Monorepo over separate repositories

Decision: A single repository with `backend/` and `frontend/` as top-level directories. **Rationale:** Single-developer project, no independent release cadence between the two halves, and cross-cutting changes (e.g., a new API field and its corresponding UI) land in one commit.

ADR-007 — Tailwind CSS, mobile-first

Decision: Tailwind CSS utility classes, written mobile-first (unprefixed utilities target small screens; `sm:` / `md:` / `lg:` are progressive enhancements for larger viewports). **Rationale:** Directly reflects NFR-1 — the primary usage pattern is a phone screen.

ADR-008 — Custom minimal roles over a membership/approval plugin

Context: The application needs multi-user support with an approval gate for new registrations. **Decision:** Two custom WordPress roles (`bp_tracker_pending` , `bp_tracker_user`) with capabilities scoped narrowly to the `bp_reading` post type via a custom `capability_type` and `map_meta_cap` , rather than installing a third-party membership plugin. **Rationale:** Minimizes third-party attack surface, keeps the authorization model fully understood and auditable, and reuses WordPress's native Users screen (filter by role, bulk "Change role to...") as the approval mechanism, instead of building a bespoke admin UI. **Trade-offs:** No polished approval UX out of the box; acceptable for the current scale (approvals happen rarely, by a single administrator).

ADR-009 — `.env` via `phpdotenv` over `wp-env` config injection

Context: With `@wordpress/env` , `wp-config.php` is generated inside an ephemeral, non-versioned environment — it cannot be edited directly, and its config injection mechanism (`.wp-env.json` config keys) is tied to the local dev tool, not

portable to a future production host. **Decision:** The plugin bootstraps a `.env` file (via `vLucas/phpdotenv`) local to `backend/`, gitignored, defining secrets only if not already defined by the hosting environment. **Rationale:** Decouples secret management from the local dev tool; portable to any future host; matches a workflow familiar from other PHP ecosystems (e.g., Laravel).

ADR-010 — Categorical X-axis over continuous time-scale X-axis for the trend chart

Context: With a true time-proportional X-axis, two readings taken hours apart on the same day render nearly on top of each other, while gaps of several days compress the rest of the chart. **Decision:** The chart's X-axis places each reading at an evenly-spaced discrete position, in chronological order, regardless of the real time elapsed between readings. The exact timestamp is shown in the tooltip on hover/tap. **Rationale:** At a twice-daily cadence, legibility matters more than time-proportional spacing; this mirrors how most consumer health-tracking apps (not analytics dashboards) render this kind of data. **Trade-offs:** The visual distance between points no longer represents elapsed time.

ADR-011 — Vitest over Jest

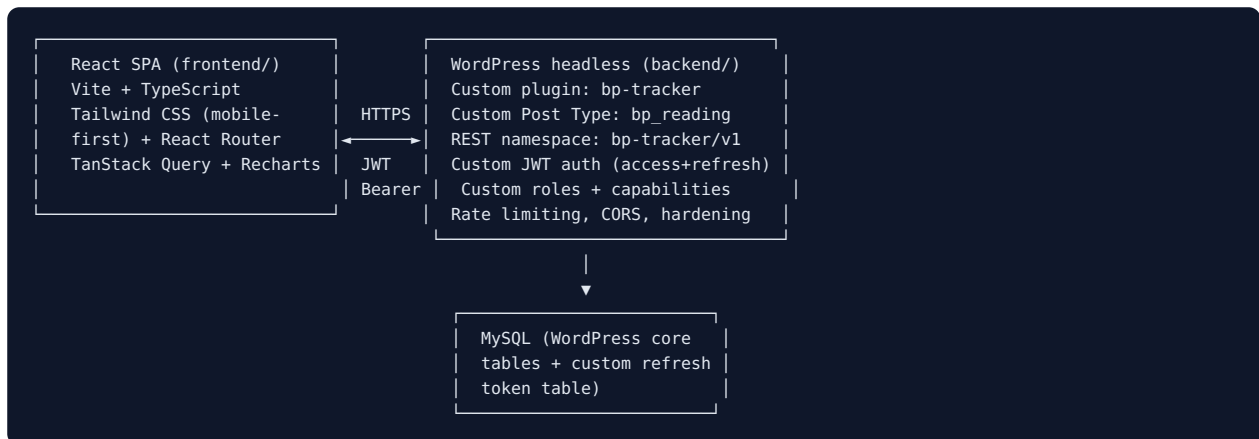
Decision: Vitest, not Jest, for the frontend test runner. **Rationale:** Native integration with Vite's build pipeline (no Babel/ts-jest configuration needed), with a largely Jest-compatible API.

ADR-012 — CI without CD

Decision: Continuous Integration (automated lint/analysis/test on every push) is implemented; Continuous Deployment is explicitly out of scope until a hosting target is chosen. **Rationale:** CI's value (catching regressions before they're committed) is independent of having a deployment target; CD without a deployment target would be inert configuration.

4. System Architecture

4.1 Component overview



Local development runs entirely via `@wordpress/env` (Docker), with the plugin directory mapped into the WordPress installation's `wp-content/plugins/` path.

4.2 Data model

`bp_reading` (Custom Post Type, not public, REST-exposed)

Meta field	Type	Required	Validation
<code>reading_datetime</code>	string (ISO 8601, with UTC offset)	yes	valid datetime
<code>systolic</code>	integer	yes	60-250
<code>diastolic</code>	integer	yes	40-150
<code>pulse</code>	integer	no	30-220
<code>weight</code>	float (kg)	no	—
<code>notes</code>	string	no	sanitized plain text

`post_author` identifies the owning user. `post_title` is generated automatically from the datetime and reading, never user-editable.

`bp_tracker_refresh_tokens` (custom table)

Column	Type	Notes
<code>id</code>	bigint, PK	
<code>user_id</code>	bigint	owner (<code>wp_users.ID</code>)
<code>family_id</code>	char(32)	groups the tokens of one login session, for reuse detection
<code>token_hash</code>	char(64), unique	SHA-256 of the token, never the raw token
<code>expires_at</code>	datetime	
<code>used_at</code>	datetime, nullable	set when the token is rotated; a second use revokes the family
<code>created_at</code>	datetime	

Expired refresh tokens are purged daily by WP-Cron. Each user also has a `bp_tracker_session_generation` user-meta counter, embedded in every access token and bumped whenever all of that user's sessions are revoked.

4.3 API surface

Namespace: `/wp-json/bp-tracker/v1`

Method	Path	Auth	Description
POST	<code>/auth/register</code>	none	Create a pending account (see section 5.2)
POST	<code>/auth/login</code>	none	Exchange credentials for an access token (response body) and a refresh token (<code>HttpOnly</code> cookie)
POST	<code>/auth/refresh</code>	refresh cookie	Rotate the refresh token and issue a new access token
POST	<code>/auth/logout</code>	refresh cookie	Revoke the current session (the token's whole family) and clear the cookie
POST	<code>/auth/logout-all</code>	refresh cookie	Revoke every session of the user, including access tokens already issued
GET	<code>/readings</code>	access token	Paginated list, filterable by <code>period_start</code> / <code>period_end</code>
POST	<code>/readings</code>	access token	Create a reading
GET	<code>/readings/{id}</code>	access token	Read a single reading (owner only)
PUT	<code>/readings/{id}</code>	access token	Update a reading (owner only)
DELETE	<code>/readings/{id}</code>	access token	Delete a reading (owner only)
GET	<code>/stats</code>	access token	Aggregate averages, min/max and count for a period

Every `/auth/*` route requires an `X-BP-Tracker-CSRF: 1` header (see section 6.1). The `access token` routes take it as `Authorization: Bearer <token>`.

5. Authorization & Roles Model

5.1 Roles

Role	Purpose	Capabilities
<code>administrator</code> (built-in)	User management only — never a data author	Full WordPress capabilities, including user management. Reserved for account administration.
<code>bp_tracker_user</code>	Standard end user	Own-scoped CRUD on <code>bp_reading</code> only (<code>edit_bp_readings</code> , <code>publish_bp_readings</code> , <code>delete_bp_readings</code> , etc.) via a custom <code>capability_type</code> + <code>map_meta_cap</code> . Explicitly not granted <code>edit_others_bp_readings</code> or <code>delete_others_bp_readings</code> .
<code>bp_tracker_pending</code>	Awaiting approval	No meaningful capabilities; blocked from authenticating even with valid credentials.

5.2 Registration & approval flow

1. A visitor registers via `POST /auth/register` (username, email, password). The account is created with role `bp_tracker_pending` .
2. `POST /auth/login` checks the authenticated user's role after credential validation; a `bp_tracker_pending` account is rejected with `403` even with correct credentials.

3. The administrator approves a pending account using WordPress's native Users screen: filtering by role, then using the built-in "Change role to..." bulk action to promote the account to `bp_tracker_user`. No custom approval interface was built — this was a deliberate choice (see ADR-008) to avoid unnecessary maintenance surface for a low-frequency, low-volume action.
4. Changing an approved account back to `bp_tracker_pending` revokes all of its sessions immediately.

5.3 Defense in depth

Authorization is enforced at two independent layers, so that a bug in one does not expose the other: 1. **Capability layer** — `bp_tracker_user` cannot act on posts it doesn't own, at the WordPress capability level (`map_meta_cap`). 2. **Application layer** — the REST controller additionally checks `post_author` against the authenticated user for every single-item operation.

5.4 Access restriction outside the API

Accounts with `bp_tracker_pending` or `bp_tracker_user` roles are redirected away from `/wp-admin` (via an `admin_init` hook and a `login_redirect` filter override) to the frontend application's origin. REST requests from these roles outside the `bp-tracker/v1` namespace are rejected with `403` (via a `rest_pre_dispatch` filter). The `administrator` role is never subject to these restrictions.

6. Security Model

6.1 Baseline protections

Area	Measure
Session model	Short-lived access token sent as a Bearer header and kept in memory; refresh token in an <code>HttpOnly; SameSite=Strict</code> cookie scoped to <code>/auth/*</code> , unreadable by JavaScript.
CSRF	The only cookie-authenticated routes (<code>/auth/*</code>) require a custom <code>X-BP-Tracker-CSRF: 1</code> header. Browsers send custom headers cross-origin only after a CORS preflight, which is granted only to the configured frontend origin.
Token revocation	Refresh tokens are stored hashed, rotated on every use, and grouped in families; reusing a spent token revokes its family. A per-user session generation lets logout-all, password changes and re-pending an account invalidate access tokens immediately.
Token signing	JWT algorithm explicitly pinned to HS256 on both encode and decode, mitigating algorithm-confusion attacks.
Authorization	Two independent layers (see 5.3).
Cross-origin requests	CORS restricted to a single configured frontend origin, never <code>*</code> , and only for the plugin's own REST namespace.
Input/output	WordPress sanitization/escaping conventions, enforced by WPCS in CI.
Static analysis	PHPStan (level 6, with WordPress-aware stub extension) catches a class of bugs — including some SQL misuse — before merge.
Secrets	Never committed; loaded via a gitignored <code>.env</code> (see ADR-009).
Least privilege	Custom roles scoped to only what's needed (see section 5).

6.2 Hardening measures

Measure	Mitigates
Rate limiting via WordPress transients: failed logins are capped per IP (5 / 15 min), per account and IP (5 / 15 min) and per account across all IPs (50 / 15 min); sign-ups at 3 / hour per IP. Bucket keys hold an HMAC of the IP or account, never the raw value.	Brute-force credential guessing, including distributed attacks on one account; fake account flooding.
SQL review of all queries against the custom refresh-token table, requiring <code>\$wpdb->prepare()</code>	SQL injection.
Dependency vulnerability scanning (<code>composer audit</code> , <code>npm audit</code>) in CI, plus Dependabot	Known CVEs in third-party packages.
<code>WP_DEBUG_DISPLAY</code> disabled (errors logged, never rendered)	Information disclosure via verbose error output.

6.3 Explicit non-goals / known limitations

This system has **not** been independently security-audited or penetration-tested. Known gaps, accepted for the project's current scope:

- No Web Application Firewall or intrusion detection.
- No automated dependency-supply-chain attestation beyond `audit` /Dependabot.
- HTTPS is assumed at the hosting layer once deployed; local development runs over plain HTTP.
- If the application is extended to store health data belonging to multiple real users, data-protection obligations for sensitive personal data (health information is a special category under most privacy regulations) should be revisited before any such rollout — this was identified during development but deliberately left outside this project's current scope.

No system is "100% secure"; the measures above reduce a known, common set of risks. Security is a continuous process, not a completed checklist item.

7. Testing Strategy

Layer	Tooling	Scope
Backend unit/integration	PHPUnit + the WordPress test suite (via <code>@wordpress/env</code> 's test environment)	CPT registration and meta validation, auth flows (login/refresh/logout/register, including negative cases), REST controller CRUD and cross-user authorization, role/capability behavior, rate limiting, JWT tampering.
Backend static analysis	PHPStan (level 6, WordPress stub extension), WPCS (<code>WordPress-Extra</code> ruleset)	Type safety, WordPress coding standards, a subset of security anti-patterns.
Frontend unit/component	Vitest + Testing Library	Auth context and token refresh behavior, form validation, chart rendering and data aggregation logic, accessibility basics (label association).
Frontend static analysis	ESLint (typescript-eslint), <code>tsc --noEmit</code>	Type safety, code quality.
Continuous Integration	GitHub Actions, one workflow per side of the stack	Every push/PR runs the full lint + analysis + test + (frontend) build pipeline; dependency audit is a separate CI step.

Every feature was required to ship with its corresponding tests in the same phase — no phase was considered complete with failing or absent tests for the functionality it introduced.

8. Manual Testing Guide

This section documents how the system was verified by hand at each stage of development, before the corresponding automated tests (and, later, the frontend) existed to exercise it.

8.1 Backend-only phase (before the frontend existed)

With `npx wp-env start` running, the API was reachable at `http://localhost:8888/wp-json/bp-tracker/v1`. All manual verification at this stage was done with `curl` (a Postman or Insomnia collection pointed at the same base URL works identically). The examples below reflect the current API: `/auth/*` routes need the `X-BP-Tracker-CSRF: 1` header, and `curl`'s cookie jar (`-c` / `-b`) carries the refresh cookie between requests.

Register:

```
curl -X POST http://localhost:8888/wp-json/bp-tracker/v1/auth/register \
-H "Content-Type: application/json" \
-H "X-BP-Tracker-CSRF: 1" \
-d '{"username":"testuser","email":"test@example.com","password":"a-strong-password"}
```

Login (expected to fail while pending, succeed once approved):

```
curl -X POST http://localhost:8888/wp-json/bp-tracker/v1/auth/login \
-c cookies.txt \
-H "Content-Type: application/json" \
-H "X-BP-Tracker-CSRF: 1" \
-d '{"username":"testuser","password":"a-strong-password"}
```

A successful response returns `access_token` (plus `expires_in` and the user) in the body and sets the refresh token as an `HttpOnly` cookie, saved here to `cookies.txt`. The access token is used as a Bearer token in subsequent requests.

Create a reading:

```
curl -X POST http://localhost:8888/wp-json/bp-tracker/v1/readings \
-H "Authorization: Bearer <access_token>" \
-H "Content-Type: application/json" \
-d '{"reading_datetime":"2026-09-13T09:00:00-03:00","systolic":120,"diastolic":80,"pulse":75}'
```

List readings, filtered by period:

```
curl "http://localhost:8888/wp-json/bp-tracker/v1/readings?period_start=2026-09-01&period_end=2026-09-30" \
-H "Authorization: Bearer <access_token>"
```

Refresh an expired access token (rotates the cookie; replaying the old one revokes the session):

```
curl -X POST http://localhost:8888/wp-json/bp-tracker/v1/auth/refresh \
-b cookies.txt -c cookies.txt \
-H "X-BP-Tracker-CSRF: 1"
```

Confirm cross-user isolation: repeat the "read/update/delete a single reading" requests using another user's access token against the first user's reading `id` — every one of these must return `403`.

8.2 Frontend-integrated phase

With both `npx wp-env start` and the frontend dev server running, manual QA followed this checklist on every significant change:

1. Log in → create a reading → confirm it appears in the history list and chart.
2. Switch between period filters (7/10/30/90 days, custom range, lifetime) and confirm the chart and summary numbers update accordingly.
3. Confirm multiple same-day readings render as distinct points, not collapsed into one.

4. Delete a reading and confirm both the list and the chart update.
5. Let the access token expire (or artificially shorten its lifetime for testing) and confirm a request transparently triggers a refresh instead of forcing a re-login.
6. Log out and confirm the refresh token is rejected on a subsequent refresh attempt.
7. Sign in on two browsers, use "sign out of all devices" on one, and confirm the other is signed out on its next request.
8. Repeat the full flow on an actual mobile device, not only a resized desktop browser window.

8.3 Roles & approval phase

1. Register a new account; confirm it cannot log in (`403` , pending message).
2. As the administrator, open the WordPress Users screen, filter by the pending role, and use "Change role to..." to promote the account.
3. Confirm the promoted account can now log in and only sees its own (empty) reading history.
4. While authenticated as a non-administrator role, attempt a request to a core WordPress REST route (e.g., `/wp-json/wp/v2/users`) and confirm `403` .
5. While authenticated as a non-administrator role, attempt to visit `/wp-admin` directly and confirm the redirect to the frontend origin.

8.4 Hardening phase

1. Submit six consecutive failed login attempts for the same account from the same origin; confirm the sixth is rejected with `429` and a `Retry-After` header, and that a correct login attempt shortly after (within the window) is still blocked.
2. Confirm a correct login shortly after a small number of failed attempts (below the threshold) succeeds and resets the counter.
3. Manually craft a JWT with `"alg": "none"` or a different algorithm than expected and confirm it is rejected.
4. Run `composer audit` and `npm audit` locally and confirm the CI steps mirror the same result.

Appendix A — Prompts Used with Claude Code

The system was built in discrete, sequentially executed phases, each specified before implementation and verified (tests passing, manual check where applicable) before moving to the next. The prompts below are reproduced in English — the language used throughout this documentation and the codebase itself — and lightly consolidated for clarity; their technical content, sequence, and intent match exactly what was used to direct Claude Code at each phase.

A note on Appendix A's history: earlier in the project, an internal policy prohibited any reference to AI development tooling anywhere in the repository (see NFR-7 in section 2.2 as it originally read, and the note under Phase A below). Under that policy, this appendix was initially written and labeled as "Phase Specifications" — the same content, framed without naming the tool used to execute it. That policy was later revisited and reversed, and this document was updated accordingly; NFR-7 and this appendix now reflect the project's current, final position. The earlier framing is mentioned here for transparency, not because it changes anything about the actual technical work described.

Phase A — Repository Bootstrap

Goal: Establish the repository structure, root-level behavioral rules, and reusable specialist guidance before any application code was written.

Specification: - Repository layout: `backend/` (WordPress plugin, mapped to `wp-content/plugins/bp-tracker` via the local dev environment), `frontend/` (Vite + React + TypeScript), `docs/` (project documentation), plus root-level configuration. - A root behavioral configuration document establishing: - All code, comments, identifiers, commit messages, and documentation in English. - Domain expertise to apply consistently: WordPress plugin architecture and security conventions; modern PHP with static analysis; React/TypeScript with Tailwind, mobile-first; technical SEO and WCAG 2.1 AA accessibility as a baseline; mandatory automated tests for every new feature; static analysis gates on both sides of the stack. - A rule prohibiting any reference to AI development tooling anywhere in the repository (commit messages, code comments, documentation, metadata fields, file/branch names). *(This rule was later revised — see the note at the start of this appendix.)* - Commit messages following Conventional Commits, in English, with no attribution

trailers. - Reusable domain-specific guidance split into three areas: WordPress/PHP conventions, frontend/React conventions, and cross-cutting code-quality requirements (testing, static analysis). - A `docs/` skeleton for ongoing architecture and API documentation.

Phase B0 — Local Environment & Plugin Scaffold

Goal: A runnable local WordPress environment and an empty, quality-tooled plugin skeleton.

Specification: - Local WordPress environment (Docker-based), PHP 8.2+, with the `backend/` directory mapped into the WordPress plugins directory, including a dedicated test environment for PHPUnit. - Plugin skeleton with a main bootstrap file and an `includes/` directory. - PHP dependency management (Composer) with development dependencies for static analysis (PHPStan, with a WordPress-aware extension) and coding-standards enforcement (WordPress Coding Standards via PHP_CodeSniffer), plus the WordPress PHPUnit polyfills needed to run tests on modern PHP. - Composer scripts for linting, auto-fixing, static analysis, and running tests. - PHPStan configured at level 6. WPCS configured against the `WordPress-Extra` ruleset with the project's text domain and function-name prefix. - A minimal smoke test confirming the test suite runs.

Phase B1 — Reading Data Model

Goal: The core domain entity, with validation and test coverage.

Specification: - A Custom Post Type for blood pressure readings: not publicly queryable, no default admin UI, REST-exposed. - Typed, REST-exposed post meta fields with sanitization and validation: a required ISO 8601 datetime, required systolic and diastolic integers within clinically plausible ranges, optional pulse within a plausible range, optional weight, optional free-text notes. - An automatically generated post title derived from the datetime and reading values (never user-editable). - Test coverage: correct CPT registration, meta fields accepted within valid ranges, meta fields rejected outside valid ranges, meta fields correctly exposed via the REST API.

Phase B2 — Authentication

Goal: Token-based authentication with real revocation, and cross-origin access control.

Specification: - A short-lived signed access token (1 hour) and a separate, database-tracked refresh token (30 days), following the access+refresh pattern (see ADR-004). The refresh token is never stored in plaintext — only its hash, in a dedicated database table created on plugin activation. - Endpoints: credential-based login issuing both tokens; token refresh (optionally rotating the refresh token); logout revoking the refresh token. - A filter authenticating REST requests via a Bearer access token, validating signature and expiry, with the signing secret sourced from an environment-specific configuration value (never hardcoded — see ADR-009), and the signing algorithm explicitly pinned (see section 6.1). - Explicit CORS configuration restricted to a single configured frontend origin, applied only to the plugin's own REST namespace. - Test coverage: successful and failed login, refresh with valid/expired/revoked tokens, logout revocation, protected-endpoint access with and without a valid token.

Phase B3 — Readings REST Controller

Goal: Full CRUD and aggregate statistics over the reading resource, correctly authorized.

Specification: - A REST controller exposing list (with period filtering and pagination), create, read, update, delete, and an aggregate statistics endpoint (average systolic/diastolic/pulse and total count for a given period). - Every endpoint requires authentication; every single-item operation additionally verifies the requesting user owns the resource (see section 5.3, defense in depth). - Responses return a clean, purpose-built representation of a reading rather than the raw WordPress post object. - Test coverage: full authenticated CRUD, cross-user access correctly rejected, validation rejecting out-of-range input, statistics correctly aggregated against known fixture data.

Phase B4 — API Documentation

Goal: A written reference for the REST surface built in B2–B3.

Specification: Document every endpoint (method, path, required headers, request body, example success and error responses) in the project's documentation.

Phase B5 — Backend Continuous Integration

Goal: Automated enforcement of the backend quality gates on every change.

Specification: A CI workflow triggered on changes under `backend/`, running dependency installation, lint, static analysis, and the test suite, failing the build on any failure.

Phase C0 — Frontend Scaffold

Goal: A runnable, quality-tooled frontend skeleton.

Specification: - A Vite-built React + TypeScript application with Tailwind CSS configured mobile-first. - Data-fetching and server-state management via a query library; a typed HTTP client automatically attaching the bearer access token. - Routing, and a charting library for the trend visualization built later. - Linting (TypeScript-aware ESLint) and formatting; strict TypeScript compiler settings. - A component test runner integrated natively with the build tool, plus a DOM-testing utility library. - A conventional directory structure separating components, pages, data-fetching hooks, cross-cutting context, and shared types.

Phase C1 — Authentication (Frontend)

Goal: A client-side implementation of the access+refresh model from Phase B2, including transparent token renewal.

Specification: - Application-wide authentication state (login, logout, authenticated flag), persisting the refresh token client-side. - An HTTP interceptor that, on receiving an authentication failure, automatically attempts a token refresh before retrying the original request, and only forces a logout if the refresh itself fails. - A login screen with clear error feedback on invalid credentials. - A route guard redirecting unauthenticated access to the login screen. - Test coverage: successful and failed login updating application state correctly, logout clearing state, the route guard redirecting when unauthenticated.

(Client-side refresh token storage was replaced right after this phase — see the next iteration.)

Iteration — Session Hardening

Goal: Take the refresh token out of JavaScript's reach and make sessions revocable and theft-evident.

Specification: - The refresh token moved from client-side storage into an `HttpOnly; SameSite=Strict` cookie scoped to the `/auth/*` routes, set and cleared by the API; the access token is kept in memory only, and the session is restored from the cookie on page load. - A required custom CSRF header on every cookie-authenticated route, relying on the strict CORS allow-list to keep it same-origin only. - Access token lifetime shortened from 1 hour to 15 minutes. - Refresh token rotation on every use, with token families: a reused (already-rotated) token revokes its whole family, and fires an action hook other code can use to log or alert. - A per-user session generation embedded in access tokens, bumped by a new "sign out of all devices" endpoint and by password changes, so already-issued access tokens stop working immediately. - Failed-login rate limiting with a `429 / Retry-After` response, and a matching lockout message and "sign out of all devices" control in the frontend. - Test coverage: cookie attributes and CSRF enforcement, rotation and reuse detection (including concurrent use of one token), revocation on logout-all and password change, rate-limit lockout.

Phase C2 — New Reading Form

Goal: The primary, most frequently used user-facing interaction.

Specification: - Data-fetching hooks for creating and listing readings, fully typed against the shared domain types. - A mobile-first form (large touch targets) covering all reading fields, client-side validating the same ranges enforced server-side, with accessible error messaging (validation errors programmatically associated with their inputs). - On successful submission, the form resets and confirms visually. - Test coverage: valid submission triggers the create call correctly; invalid submission is blocked client-side and shows an error without calling the API; form fields are properly labeled (a basic accessibility check).

Phase C3 — History Dashboard (initial version)

Goal: A view into the accumulated data: list, trend visualization, and summary.

Specification: - A history view listing readings most-recent-first, responsively switching between a stacked-card layout on small screens and a denser layout on larger ones. - A trend chart plotting systolic and diastolic values over time, with

quick period filters adjusting the underlying query. - A summary panel showing period averages via the aggregate statistics endpoint. - Per-row deletion with a confirmation step. - Test coverage: correct rendering against fixture data, period filters triggering correctly parameterized queries, deletion requiring and respecting confirmation.

Phase C4 — Frontend Continuous Integration

Goal: Automated enforcement of the frontend quality gates on every change.

Specification: A CI workflow triggered on changes under `frontend/`, running dependency installation, lint, type-checking, tests, and a production build, failing on any failure.

Iteration — Secrets Configuration Revisited

Goal: Replace the initial local-dev-tool-specific secrets mechanism with a portable one.

Specification: As described in ADR-009 — the JWT signing secret and CORS origin configuration were migrated from a dev-tool-specific override mechanism to a `.env`-based approach, loaded by the plugin itself, gitignored, and falling back gracefully to any pre-existing hosting-level configuration rather than overriding it.

Iteration — Trend Chart Redesign

Goal: Correctly and legibly visualize a variable, multiple-per-day reading cadence.

Specification: The chart's X-axis was changed from a continuous, time-proportional scale to a categorical (ordinal) scale, so that multiple same-day readings render as distinct, evenly-spaced points rather than visually overlapping (see ADR-010). A day-boundary marker and a full-precision tooltip were added to preserve exact-time information without sacrificing legibility.

Iteration — Dashboard Expansion (period selector, zoom, clinical context, summary trends)

Goal: A more complete view into accumulated data, addressing legibility at scale and providing clinical reference context.

Specification: - A period selector offering multiple preset ranges, a custom date-range option, and a lifetime (all-data) option. - Interactive zoom/pan within the currently loaded range, in addition to the preset filters. - Clinical reference context: rather than shared background zones (which was found to be visually and numerically ambiguous across two metrics with different threshold values on a shared axis — see the visual redesign iteration below), reference lines at the commonly cited normal thresholds, and a per-reading category shown in the tooltip and in a period-level distribution summary. - Automatic aggregation to a daily average once the number of points in the selected range exceeds a legibility threshold, with an explicit control to reveal every individual reading regardless. - A period-over-period comparison in the summary panel (current period's averages against the immediately preceding period of equal length).

Iteration — Visual Redesign

Goal: Correct a data-visualization ambiguity and bring the interface to a level of visual polish matching the rest of the codebase's engineering standards.

Specification: - The two metrics (systolic, diastolic), initially rendered as two separate charts, were unified into a single chart sharing one Y-axis, with each series distinctly colored. - The clinical background-zone shading from the initial implementation was replaced with per-metric reference lines and point-level category labels (see the dashboard expansion iteration above), since a single shared background cannot correctly represent two metrics with different clinical thresholds at the same time. - A full visual pass across the dashboard components: consistent card treatment, typographic hierarchy, a proper color legend, dark-mode support via the operating system's color-scheme preference, and layout adjustments to make effective use of large viewports rather than a narrow, centered column regardless of available screen width.

Phase D1 — User Roles & Registration

Goal: Multi-user support with a minimal-privilege model and an approval gate (see section 5).

Specification: - Two custom roles registered on plugin activation (and cleanly removed on deactivation): a pending role with no meaningful capabilities, and a standard user role scoped — via a custom post-type capability type and WordPress's meta-capability mapping — to only the reading resource, explicitly excluding the capability to act on another user's resources. - The reading Custom Post Type reconfigured to use this custom capability type, activating the granular permission model. - A public registration endpoint creating a new account in the pending role, with uniqueness and basic input validation, returning no token (registration alone does not grant access). - The login endpoint extended to reject a pending account even with correct credentials, with a clear status explaining the account requires approval. - Test coverage: registration correctly creates a pending account and rejects invalid/duplicate input; a pending account cannot log in; an approved account logs in normally; the standard role correctly lacks the capability to act on another user's data.

Phase D2 — Access Restriction Outside the API

Goal: Ensure non-administrator roles cannot reach any WordPress surface beyond the application's own API (see section 5.4).

Specification: - A redirect intercepting any direct administrative-area access attempt by a non-administrator role, sending the user to the frontend application's configured origin instead — covering both direct navigation and the native WordPress login form's post-login redirect. - A REST-layer filter rejecting, with an explicit forbidden response, any request from a non-administrator role targeting a route outside the application's own REST namespace — while leaving unauthenticated requests and administrator access unaffected. - Documentation of the resulting role and approval model in the project's architecture documentation. - Test coverage: a non-administrator role is correctly rejected on an out-of-namespace route; an administrator is unaffected; the access-restriction decision logic is isolated into a testable unit separate from the actual redirect side-effect (which cannot itself be unit-tested).

Phase — Historical Data Migration

Goal: Replace placeholder development data with the author's own historical readings, without ever committing personal health data to version control.

Specification: Existing placeholder data was removed. Historical readings were inserted using the exact same validated code path as the public API (rather than a raw database insert bypassing validation), executed as the application's standard end-user role, via a script run once, outside the repository, and confirmed afterward to have left no trace in version control. This phase has no reproducible prompt included here — not due to the AI-attribution policy discussed above, but because doing so would require embedding personal health data (specific readings, dates, and a symptom note) in project documentation, which this phase was specifically designed to avoid.

Phase E1 — Rate Limiting

Goal: Mitigate credential brute-forcing and fake-account flooding.

Specification: One rate limiter shared by the login and registration endpoints, built on WordPress's transient storage (no new infrastructure dependency). Login failures are counted per IP, per account and IP, and per account across all IPs (a looser cap against distributed attacks); sign-ups per IP. Bucket keys hold a keyed hash (HMAC) of the IP or account rather than the raw value, the client IP honours only explicitly trusted proxies, exceeding a limit returns a standard `429 / Retry-After` response, and a successful login clears the relevant counters. Test coverage: each limit is enforced and correctly scoped, and a legitimate login before the threshold is reached both succeeds and resets the counters.

Phase E2 — Token Algorithm Pinning

Goal: Close a known class of JWT library misconfiguration vulnerability (algorithm-confusion attacks).

Specification: Both signing and verification explicitly pin a single algorithm, never trusting an algorithm value carried by the token itself. Test coverage: a token forged with a different or absent algorithm declaration is rejected.

Phase E3 — SQL Review

Goal: Confirm no raw SQL interpolation exists against the custom refresh-token table.

Specification: A manual review of every query against the custom table, confirming parameterized queries throughout, paired with a static-analysis re-run.

Phase E4 — Dependency Vulnerability Scanning

Goal: Catch known vulnerabilities in third-party dependencies before they reach production.

Specification: A dependency-audit script on each side of the stack, wired into the corresponding CI workflow as a failing check on high/critical findings, plus automated dependency-update proposals via a scheduled bot.

Phase E5 — Safe Error Reporting

Goal: Prevent verbose error output from disclosing implementation details.

Specification: Debug logging enabled locally, but error *display* explicitly disabled, with the distinction documented for any future non-local deployment.

Appendix B — Glossary

Term	Meaning
ADR	Architecture Decision Record — a short document capturing a single architecture decision, its context, and its trade-offs.
CPT	Custom Post Type — a WordPress mechanism for defining content types beyond posts/pages.
WPCS	WordPress Coding Standards — the official PHP coding-standard ruleset for WordPress code.
JWT	JSON Web Token — a compact, signed token format used here for API authentication.
CORS	Cross-Origin Resource Sharing — the browser mechanism controlling which origins may call an API.
<code>map_meta_cap</code>	A WordPress mechanism translating broad capability checks into fine-grained, per-object ones.
CI	Continuous Integration — automated build/test/lint execution on every code change.
CD	Continuous Deployment — automated release to a live environment (not implemented in this project; see ADR-012).

This document was written as part of the project's own development process and is maintained alongside the codebase.